Web scraping avec Python

Lino Galiana
Version du 16 octobre 2026

`Python` permet de facilement récupérer une page web pour en extraire des données à restructurer. Le *web scraping*, que les Canadiens nomment *« moissonnage du web »*, est une manière de plus en plus utilisée de récupérer une grande masse d'information en temps réel. Ce chapitre présente les deux principaux paradigmes par le biais de `BeautifulSoup` et `Selenium` et les principaux défis du *web scraping*.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Enjeux **2**
1.1 La zone grise de la légalité du *web scraping* 2
1.2 Stabilité et fiabilité des informations reçues 2
1.3 Les bonnes pratiques 3
2 Un détour par le Web : comment fonctionne un site ? **3**
2.1 Les balises 4
3 Scraper avec `Python`: le package `BeautifulSoup` **5**
3.1 Les packages disponibles 5
3.2 Récupérer le contenu d'une page HTML 6
3.3 La méthode `find` 7
3.4 La méthode `find_all` 7
4 Exercice guidé : obtenir la liste des équipes de Ligue 1 ..... **8**
5 Pour aller plus loin **11**
5.1 Récupération des localisations des stades 11
5.2 Carte des stades avec `folium` 15
6 Récupérer des informations sur les pokemons **15**
6.1 Version non guidée 15
6.2 Version guidée 16
7 `Selenium` : mimer le comportement d'un utilisateur internet **18**
7.1 Premier exemple en scrapant un moteur de recherche 18
7.2 Exercice supplémentaire 21
Informations additionnelles 21

Compétences à l'issue de ce chapitre

- Comprendre les enjeux du web scraping, en particulier les questions de légalité (RGPD, zone grise), de stabilité des sites, et de fiabilité des données ;
- Appliquer de bonnes pratiques lors du scraping : consulter le fichier `robots.txt`, espacer les requêtes, éviter de surcharger les sites, privilégier les périodes creuses ;
- Saisir la structure HTML d'une page (balises, parent-enfant) pour cibler correctement les éléments à extraire ;
- Utiliser le package `requests` pour récupérer le contenu d'une page web, et `BeautifulSoup` pour analyser et naviguer dans le HTML (méthodes `find`, `find_all`) ;
- Mettre en pratique le scraping avec un exercice concret (liste des équipes de Ligue 1) ;

- Découvrir Selenium pour simuler le comportement d'un utilisateur sur des pages dynamiques générées par JavaScript ;
- Évaluer les limites du web scraping et justifier l'usage d'API plus robustes quand elles sont disponibles.

Le *web scraping* désigne les techniques d'extraction du contenu des sites internet. C'est une pratique très utile pour toute personne souhaitant travailler sur des informations disponibles en ligne, mais n'existant pas forcément sous la forme d'un tableau *Excel*.

Ce TP vous présente comment créer et exécuter des robots afin de recupérer rapidement des informations utiles à vos projets actuels ou futurs. Il part de quelques cas d'usages concret. Ce chapitre est très fortement inspiré et réadapté à partir de celui de Xavier Dupré, l'ancien professeur de la matière.

1 Enjeux

Un certain nombre d'enjeux du *web scraping* ne seront évoqués que superficiellement dans le cadre de ce chapitre.

1.1 La zone grise de la légalité du *web scraping*

En premier lieu, en ce qui concerne la question de la légalité de la récupération d'information par *scraping*, il existe une zone grise. Ce n'est pas parce qu'une information est disponible sur internet, directement ou avec un peu de recherche, qu'elle peut être récupérée et réutilisée.

L'excellent cours d'Antoine Palazzolo évoque un certain nombre de cas médiatiques et judiciaires sur cette question. Dans le champ français, la CNIL a publié en 2020 de nouvelles directives sur le *web scraping* reprécisant que toute donnée ne peut être réutilisée à l'insu de la personne à laquelle ces données appartiennent. Autrement dit, en principe, les données collectées par *web scraping* sont soumises au RGPD, c'est-à-dire nécessitent le consentement des personnes à partir desquelles la réutilisation des données est faite.

Il est donc recommandé d'**être vigilant avec les données récupérées** par *web scraping* pour ne pas se mettre en faute légalement.

1.2 Stabilité et fiabilité des informations reçues

La récupération de données par *web scraping* est certes pratique mais elle ne correspond pas nécessairement à un usage pensé, ou désiré, par un fournisseur de données. Les données étant coûteuses à collecter et à mettre à disposition, certains sites ne désirent pas nécessairement que celles-ci soient extraites gratuitement et facilement. *A fortiori* lorsque la donnée peut permettre à un concurrent de disposer d'une information utile d'un point de vue commercial (prix d'un produit concurrent, etc.).

Les acteurs mettent donc souvent en oeuvre des stratégies pour bloquer ou limiter la quantité de données scrapées. La méthode la plus classique est la détection et le blocage des requêtes faites par des robots plutôt que par des humains. Pour des acteurs spécialisés, cette détection est très facile car de nombreuses preuves permettent d'identifier si une visite du site *web* provient d'un utilisateur humain derrière un navigateur ou d'un robot. Pour ne citer que quelques indices : vitesse de la navigation entre pages, rapidité à extraire la donnée, empreinte digitale du navigateur utilisé, capacité à répondre à des questions aléatoires (captcha)... Les bonnes pratiques, évoquées par la suite, ont pour objectif de faire en sorte qu'un robot se comporte de manière civile en adoptant un comportement proche de celui de l'humain mais sans contrefaire le fait qu'il ne s'agit pas d'un humain.

Il convient d'ailleurs d'être prudent quant aux informations reçues par *web scraping*. La donnée étant au coeur du modèle économique de certains acteurs, certains n'hésitent pas à renvoyer des données fausses aux robots plutôt que les bloquer. C'est de bonne guerre ! Une autre technique piège s'appelle le *honey pot*. Il s'agit de pages qu'un humain n'irait jamais visiter - par exemple parce qu'elles n'apparaissent pas dans l'interface graphique - mais sur lesquelles un robot, en recherche automatique de contenu, va rester bloquer.

Sans aller jusqu'à la stratégie de blocage du *web scraping*, d'autres raisons peuvent expliquer qu'une récupération de données ait fonctionné par le passé mais ne fonctionne plus. La plus fréquente est un changement dans la structure d'un site *web*. Le *web scraping* présente en effet l'inconvénient d'aller chercher de l'information dans une structure très hiérarchisée. Un changement dans cette structure peut suffire à rendre un robot incapable de récupérer du contenu. Or, pour rester attractifs, les sites *web* changent fréquemment ce qui peut facilement rendre inopérant un robot.

De manière générale, l'un des principaux messages de ce chapitre, à retenir, est que le **web scraping est une solution de dernier ressort, pour des récupérations ponctuelles de données sans garantie de fonctionnement ultérieur**. Il est préférable de **privilégier les API lorsque celles-ci sont disponibles**. Ces dernières ressemblent à un contrat (formel ou non) entre un fournisseur de données et un utilisateur où sont définis des besoins (les données) mais aussi des conditions d'accès (nombre de requêtes, volumétrie, authentification...) là où le *web scraping* est plus proche du comportement dans le *Far West*.

1.3 Les bonnes pratiques

La possibilité de récupérer des données par l'intermédiaire d'un robot ne signifie pas qu'on peut se permettre de ne pas être civilisé. En effet, lorsqu'il est non-maîtrisé, le *web scraping* peut ressembler à une attaque informatique classique pour faire sauter un site *web* : le déni de service. Le cours d'Antoine Palazzolo revient sur certaines bonnes pratiques qui ont émergé dans la communauté des *scrapeurs*. Il est recommandé de lire cette ressource pour en apprendre plus sur ce sujet. Y sont évoquées plusieurs conventions, parmi lesquelles :

- Se rendre, depuis la racine du site, sur le fichier `robots.txt` pour vérifier les consignes proposées par les développeurs du site *web* pour cadrer le comportement des robots ;
- Espacer chaque requêtes de plusieurs secondes, comme le ferait un humain, afin d'éviter de surcharger le site *web* et de le faire sauter par déni de service ;
- Faire les requêtes dans les heures creuses de fréquentation du site *web* s'il ne s'agit pas d'un site consulté internationalement. Par exemple, pour un site en français, lancer le robot pendant la nuit en France métropolitaine, est une bonne pratique. Pour lancer un robot depuis `Python` à une heure programmée à l'avance, il existe les `cronjobs`.

2 Un détour par le Web : comment fonctionne un site ?

Même si ce TP ne vise pas à faire un cours de web, il vous faut néanmoins certaines bases sur la manière dont un site internet fonctionne afin de comprendre comment sont structurées les informations sur une page.

Un site Web est un ensemble de pages codées en *HTML* qui permet de décrire à la fois le contenu et la forme d'une page *Web*.

Pour voir cela, ouvrez n'importe quelle page web et faites un clic-droit dessus.

- Sous `Chrome` : Cliquez ensuite sur *« Affichez le code source de la page »* (CTRL+U) ;
- Sous `Firefox` : *« Code source de la page »* (CTRL+MAJ+K) ;
- Sous `Edge` : *« Affichez la page source »* (CTRL+U) ;
- Sous `Safari` : voir comment faire ici

Si vous savez quel élément vous intéresse, vous pouvez également ouvrir l'inspecteur du navigateur (clic droit sur l'élément + « Inspecter »), pour afficher les balises encadrant votre élément de façon plus ergonomique, un peu comme un zoom.

2.1 Les balises

Sur une page web, vous trouverez toujours à coup sûr des éléments comme `<head>`, `<title>`, etc. Il s'agit des codes qui vous permettent de structurer le contenu d'une page *HTML* et qui s'appellent des **balises**. Citons, par exemple, les balises `<p>`, `<h1>`, `<h2>`, `<h3>`, `<strong>` ou `<em>`. Le symbole `< >` est une balise : il sert à indiquer le début d'une partie. Le symbole `</ >` indique la fin de cette partie. La plupart des balises vont par paires, avec une *balise ouvrante* et une *balise fermante* (par exemple `<p>` et `</p>`).

Par exemple, les principales balises définissant la structure d'un tableau sont les suivantes :

Balise	Description
`<table>`	Tableau
`<caption>`	Titre du tableau
`<tr>`	Ligne de tableau
`<th>`	Cellule d'en-tête
`<td>`	Cellule
`<thead>`	Section de l'en-tête du tableau
`<tbody>`	Section du corps du tableau
`<tfoot>`	Section du pied du tableau

2.1.a Application : un tableau en HTML

Le code `HTML` du tableau suivant :

```
<table>
    <caption> Le Titre de mon tableau </caption>
    <tr>
        <th>Nom</th>
        <th>Profession</th>
    </tr>
    <tr>
        <td>Astérix</td>
        <td></td>
    </tr>
    <tr>
        <td>Obélix</td>
        <td>Tailleur de Menhir</td>
    </tr>
</table>
```

Donnera dans le navigateur :

Nom	Profession
Astérix	

Obélix	Tailleur de Menhir

2.1.b Parent et enfant

Dans le cadre du langage HTML, les termes de parent (*parent*) et enfant (*child*) servent à désigner des élements emboîtés les uns dans les autres. Dans la construction suivante, par exemple :

```
<div>
    <p>
       bla,bla
    </p>
</div>
```

Sur la page web, cela apparaitra de la manière suivante :

On dira que l'élément `<div>` est le parent de l'élément `<p>` tandis que l'élément `<p>` est l'enfant de l'élément `<div>`.

> *Mais pourquoi apprendre ça pour « scraper » ?*

Parce que, pour bien récupérer les informations d'un site internet, il faut pouvoir comprendre sa structure et donc son code HTML. Les fonctions `Python` qui servent au *scraping* sont principalement construites pour vous permettre de naviguer entre les balises. Avec `Python`, vous allez en fait reproduire votre comportement manuel de recherche de manière à l'automatiser.

3 Scraper avec `Python`: le package `BeautifulSoup`

3.1 Les packages disponibles

Dans la première partie de ce chapitre, nous allons essentiellement utiliser le package `BeautifulSoup4`, en conjonction avec `requests`. Ce dernier *package* permet de récupérer le texte brut d'une page qui sera ensuite inspecté via `BeautifulSoup4`.

`BeautifulSoup` sera suffisant quand vous voudrez travailler sur des pages HTML statiques. Dès que les informations que vous recherchez sont générées via l'exécution de scripts Javascript, il vous faudra passer par des outils comme Selenium.

De même, si vous ne connaissez pas l'URL, il faudra passer par un *framework* comme Scrapy, qui passe facilement d'une page à une autre. On appelle cette technique le *« web crawling »*. `Scrapy` est plus complexe à manipuler que `BeautifulSoup` : si vous voulez plus de détails, rendez-vous sur la page du tutoriel `Scrapy`.

Le *web scraping* est un domaine où la reproductibilité est compliquée à mettre en oeuvre. Une page *web* évolue potentiellement régulièrement et d'une page web à l'autre, la structure peut être très différente ce qui rend certains codes difficilement exportables. Par conséquent, la meilleure manière d'avoir un programme fonctionnel est de comprendre la structure d'une page web et dissocier les éléments exportables à d'autres cas d'usages des requêtes *ad hoc*.

```
!pip install lxml
!pip install bs4
```

> **Note**
>
> Pour être en mesure d'utiliser `Selenium`, il est nécessaire de faire communiquer `Python` avec un navigateur *web* (Firefox ou Chromium). Le *package* `webdriver-manager` permet de faire savoir à `Python` où se trouve ce navigateur s'il est déjà installé dans un chemin standard. Pour l'installer, le code de la cellule ci-dessous peut être utilisé.

Pour faire fonctionner `Selenium`, il faut utiliser un package nommé `webdriver-manager`. On va donc l'installer, ainsi que `selenium` :

```
!pip install selenium
!pip install webdriver-manager
```

3.2 Récupérer le contenu d'une page HTML

On va commencer doucement. Prenons une page *wikipedia*, par exemple celle de la Ligue 1 de football, millésime 2019-2020 : Championnat de France de football 2019-2020. On va souhaiter récupérer la liste des équipes, ainsi que les url des pages Wikipedia de ces équipes.

Etape 1️⃣ : se connecter à la page wikipedia et obtenir le code source. Pour cela, le plus simple est d'utiliser le package `requests`. Celui-ci permet, au niveau de `Python` de faire la requête HTTP adéquate pour avoir le contenu d'une page à partir de son URL:

```
import requests
url_ligue_1 = "https://fr.wikipedia.org/wiki/Championnat_de_France_de_football_2019-2020"

request_text = requests.get(
    url_ligue_1,
    headers={"User-Agent": "Python for data science tutorial"}
).content
```

```
request_text[:150]
```

```
b'<!DOCTYPE html>\n<html class="client-nojs vector-feature-language-in-header-enabled vector-
feature-language-in-main-menu-disabled vector-feature-langua'
```

> **Warning**
>
> Pour limiter le volume de *bot* récupérant les informations depuis Wikipedia (très utilisé par exemple par les LLM), il faut dorénavant indiquer un *user agent* par le biais de `request`. C'est d'ailleurs une bonne pratique qui permet aux sites de connaître les consommateurs de ses ressources.

Etape 2️⃣ : rechercher, dans ce code source foisonnant, les balises qui permettent d'extraire l'information qui nous intéresse. C'est l'intérêt principal du package `BeautifulSoup` que d'offrir des méthodes simples d'usage pour chercher, dans des textes pourtant complexes, des chaines de caractères à partir de balises HTML ou XML.

```
import bs4
page = bs4.BeautifulSoup(request_text, "lxml")
```

Si on *print* l'objet `page` créée avec `BeautifulSoup`, on voit que ce n'est plus une chaine de caractères mais bien une page HTML avec des balises. On peut à présent chercher des élements à l'intérieur de ces balises.

3.3 La méthode `find`

Comme première illustration de la puissance de `BeautifulSoup`, on veut connaître le titre de la page. Pour cela, on utilise la méthode `.find` et on lui demande *« title »*

```
print(page.find("title"))
```

```
<title>Championnat de France de football 2019-2020 — Wikipédia</title>
```

La méthode `.find` ne renvoie que la première occurrence de l'élément.

Pour vous en assurer, vous pouvez :

- copier le bout de code source obtenu lorsque vous cherchez une `table`,
- le coller dans une cellule de votre notebook,
- et passer la cellule en *« Markdown »*.

Si on reprend le code précédent et remplace `title` par `table`, cela nous donne

```
print(page.find("table"))
```

ce qui est le texte source permettant de générer le tableau suivant :

Généralités Sport Football Organisateur(s) LFP Édition 82e Lieu(x) France et Monaco Date Du 9 août 2019au 8 mars 2020 (arrêt définitif) Participants 20 équipes Matchs joués 279 (sur 380 prévus) Site web officiel Site officiel

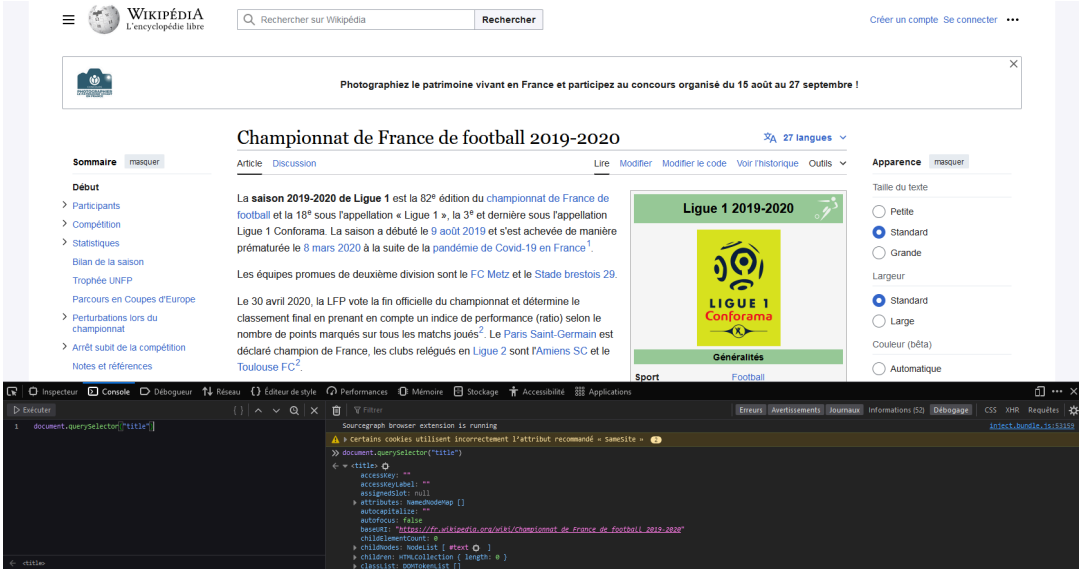
3.4 La méthode `find_all`

Pour trouver toutes les occurrences, on utilise `.find_all()`.

```
print("Il y a", len(page.find_all("table")), "éléments dans la page qui sont des <table>")
```

```
Il y a 34 éléments dans la page qui sont des <table>
```

> **Tip**
>
> `Python` n'est pas le seul langage qui permet de récupérer des éléments issus d'une page web. C'est l'un des objectifs principaux de `Javascript`, qui est accessible par le biais de n'importe quel navigateur web.
>
> Par exemple, pour faire le parallèle avec `page.find('title')` que nous avons utilisé au niveau de `Python`, vous pouvez ouvrir la page précédemment mentionnée avec votre navigateur. Après avoir ouvert les outils de développement du navigateur (CTRL+MAJ+K sur `Firefox`), vous pouvez taper dans la console `document.querySelector("title")` qui vous permettra d'obtenir le contenu du noeud HTML recherché:



Si vous êtes amenés à utiliser `Selenium` pour faire du *web scraping*, vous retrouverez, en fait, ces verbes `Javascript` dans n'importe quelle méthode que vous allez utiliser.

La compréhension de la structure d'une page et de l'interaction de celle-ci avec le navigateur est extrêmement utile lorsqu'on fait du *scraping*, y compris lorsque le site est purement statique, c'est-à-dire qu'il ne comporte pas d'éléments réagissant à une action d'un navigateur web.

4 Exercice guidé : obtenir la liste des équipes de Ligue 1

Dans le premier paragraphe de la page « *Participants* », on a le tableau avec les résultats de l'année.

> Exercice 1 : Récupérer les participants de la Ligue 1
>
> Pour cela, nous allons procéder en 6 étapes:
>
> 1. Trouver le tableau
> 2. Récupérer chaque ligne du tableau
> 3. Nettoyer les sorties en ne gardant que le texte sur une ligne
> 4. Généraliser sur toutes les lignes
> 5. Récupérer les entêtes du tableau
> 6. Finalisation du tableau

1️⃣ Trouver le tableau

```
# on identifie le tableau en question : c'est le premier qui a cette classe "wikitable sortable"
tableau_participants = page.find('table', {'class' : 'wikitable sortable'})
```

```
print(tableau_participants)
```

2️⃣ Récupérer chaque ligne du tableau

On recherche d'abord toutes les lignes du tableau avec la balise `tr`

```
table_body = tableau_participants.find('tbody')
rows = table_body.find_all('tr')
```

On obtient une liste où chaque élément est une des lignes du tableau Pour illustrer cela, on va d'abord afficher la première ligne. Celle-ci correspond aux entêtes de colonne:

```
print(rows[0])
```

La seconde ligne va correspondre à la ligne du premier club présent dans le tableau :

```
print(rows[1])
```

3️⃣ Nettoyer les sorties en ne gardant que le texte sur une ligne

On va utiliser l'attribut `text` afin de se débarrasser de toute la couche de HTML qu'on obtient à l'étape 2.

Un exemple sur la ligne du premier club :

- on commence par prendre toutes les cellules de cette ligne, avec la balise `td`.
- on fait ensuite une boucle sur chacune des cellules et on ne garde que le texte de la cellule avec l'attribut `text`.
- enfin, on applique la méthode `strip()` pour que le texte soit bien mis en forme (sans espace inutile etc).

```
cols = rows[1].find_all('td')
print(cols[0])
print(cols[0].text.strip())
```

```
<td     id="mwTA"><a     href="https://fr.wikipedia.org/wiki/Paris_Saint-Germain_Football_Club"
id="mwTQ" rel="mw:WikiLink" title="Paris Saint-Germain Football Club">Paris Saint-Germain</a></
td>
Paris Saint-Germain
```

```
for ele in cols :
    print(ele.text.strip())
```

```
Paris Saint-Germain
1974
637
1er
Thomas Tuchel
2018
Parc des Princes
47 929
46
```

4️⃣ Généraliser sur toutes les lignes :

```
for row in rows:
    cols = row.find_all('td')
    cols = [ele.text.strip() for ele in cols]
```

On a bien réussi à avoir les informations contenues dans le tableau des participants du championnat. Mais la première ligne est étrange : c'est une liste vide ...

Il s'agit des en-têtes : elles sont reconnues par la balise `th` et non `td`.

On va mettre tout le contenu dans un dictionnaire, pour le transformer ensuite en DataFrame pandas :

```
dico_participants = dict()
for row in rows:
    cols = row.find_all('td')
    cols = [ele.text.strip() for ele in cols]
    if len(cols) > 0 :
        dico_participants[cols[0]] = cols[1:]

dico_participants
```

```
import pandas as pd
data_participants = pd.DataFrame.from_dict(dico_participants,orient='index')
data_participants.head()
```

	0	1	2	3	4	5	6	7
Paris Saint-Germain	1974	637	1er	Thomas Tuchel	2018	Parc des Princes	47 929	46
LOSC Lille	2000	120	2e	Christophe Galtier	2017	Stade Pierre-Mauroy	49 712	59
Olympique lyonnais	1989	310	3e	Rudi Garcia	2019	Groupama Stadium	57 206	60
AS Saint-Étienne	2004	100	4e	Claude Puel	2019	Stade Geoffroy-Guichard	41 965	66
Olympique de Marseille	1996	110	5e	André Villas-Boas	2019	Orange Vélodrome	66 226	69

5️⃣ Récupérer les en-têtes du tableau :

```
for row in rows:
    cols = row.find_all('th')
    print(cols)
    if len(cols) > 0 :
        cols = [ele.get_text(separator=' ').strip().title() for ele in cols]
        columns_participants = cols
```

```
columns_participants
```

```
['Club',
 'Dernière Montée',
 'Budget [ 3 ] En M €',
 'Classement 2018-2019',
 'Entraîneur',
 'Depuis',
 'Stade',
 'Capacité En L1 [ 4 ]',
 'Nombre De Saisons En L1']
```

6️⃣ Finalisation du tableau

```
data_participants.columns = columns_participants[1:]
```

```
data_participants.head()
```

	Dernière Montée	Budget [3] En M €	Classement 2018-2019	Entraîneur	Depuis	Stade	Capacité En L1 [4]	Nombre De Saisons En L1
Paris Saint-Germain	1974	637	1er	Thomas Tuchel	2018	Parc des Princes	47 929	46
LOSC Lille	2000	120	2e	Christophe Galtier	2017	Stade Pierre-Mauroy	49 712	59
Olympique lyonnais	1989	310	3e	Rudi Garcia	2019	Groupama Stadium	57 206	60
AS Saint-Étienne	2004	100	4e	Claude Puel	2019	Stade Geoffroy-Guichard	41 965	66
Olympique de Marseille	1996	110	5e	André Villas-Boas	2019	Orange Vélodrome	66 226	69

5 Pour aller plus loin

5.1 Récupération des localisations des stades

Essayez de comprendre pas à pas ce qui est fait dans les étapes qui suivent (la récupération d'informations supplémentaires en naviguant dans les pages des différents clubs).

```
import requests
import bs4
import pandas as pd
from urllib.parse import urljoin


def retrieve_page(url: str) -> bs4.BeautifulSoup:
    """
    Retrieves and parses a webpage using BeautifulSoup.

    Args:
        url (str): The URL of the webpage to retrieve.

    Returns:
        bs4.BeautifulSoup: The parsed HTML content of the page.
    """
    r = requests.get(url, headers={"User-Agent": "Python for data science tutorial"})
    page = bs4.BeautifulSoup(r.content, 'html.parser')
    return page


def extract_team_name_url(team: bs4.element.Tag) -> dict:
    """
    Extracts the team name and its corresponding Wikipedia URL.

    Args:
        team (bs4.element.Tag): The BeautifulSoup tag containing the team information.

    Returns:
        dict: A dictionary with the team name as the key and the Wikipedia URL as the value, or
None if not found.
    """
    try:
        team_url = team.find('a').get('href')
        equipe = team.find('a').get('title')
        url_get_info = urljoin("http://fr.wikipedia.org", team_url)
        print(f"Retrieving information for {equipe}")
        return {equipe: url_get_info}
    except AttributeError:
        print(f"No <a> tag for \"{team}\"")
        return None
```

```
def explore_team_page(wikipedia_team_url: str) → bs4.BeautifulSoup:
    """
    Retrieves and parses a team's Wikipedia page.

    Args:
        wikipedia_team_url (str): The URL of the team's Wikipedia page.

    Returns:
        bs4.BeautifulSoup: The parsed HTML content of the team's Wikipedia page.
    """
    r = requests.get(
        wikipedia_team_url, headers={"User-Agent": "Python for data science tutorial"}
    )
    page = bs4.BeautifulSoup(r.content, 'html.parser')
    return page


def extract_stadium_info(search_team: bs4.BeautifulSoup) → tuple:
    """
    Extracts stadium information from a team's Wikipedia page.

    Args:
        search_team (bs4.BeautifulSoup): The parsed HTML content of the team's Wikipedia page.

    Returns:
         tuple: A tuple containing the stadium name, latitude, and longitude, or (None, None,
None) if not found.
    """
    for stadium in search_team.find_all('tr'):
        try:
            header = stadium.find('th', {'scope': 'row'})
            if header and header.contents[0].string == "Stade":
                name_stadium, url_get_stade = extract_stadium_name_url(stadium)
                if name_stadium and url_get_stade:
                    latitude, longitude = extract_stadium_coordinates(url_get_stade)
                    return name_stadium, latitude, longitude
        except (AttributeError, IndexError) as e:
            print(f"Error processing stadium information: {e}")
    return None, None, None


def extract_stadium_name_url(stadium: bs4.element.Tag) → tuple:
    """
    Extracts the stadium name and URL from a stadium element.

    Args:
        stadium (bs4.element.Tag): The BeautifulSoup tag containing the stadium information.

    Returns:
         tuple: A tuple containing the stadium name and its Wikipedia URL, or (None, None) if
not found.
    """
    try:
        url_stade = stadium.find_all('a')[1].get('href')
        name_stadium = stadium.find_all('a')[1].get('title')
        url_get_stade = urljoin("http://fr.wikipedia.org", url_stade)
```

```
        return name_stadium, url_get_stade
    except (AttributeError, IndexError) as e:
        print(f"Error extracting stadium name and URL: {e}")
        return None, None


def extract_stadium_coordinates(url_get_stade: str) -> tuple:
    """
    Extracts the coordinates of a stadium from its Wikipedia page.

    Args:
        url_get_stade (str): The URL of the stadium's Wikipedia page.

    Returns:
        tuple: A tuple containing the latitude and longitude of the stadium, or (None, None) if
not found.
    """
    try:
        soup_stade = retrieve_page(url_get_stade)
        kartographer = soup_stade.find('a', {'class': "mw-kartographer-maplink"})
        if kartographer:
            coordinates = kartographer.get('data-lat') + "," + kartographer.get('data-lon')
            latitude, longitude = coordinates.split(",")
            return latitude.strip(), longitude.strip()
        else:
            return None, None
    except Exception as e:
        print(f"Error extracting stadium coordinates: {e}")
        return None, None


def extract_team_info(url_team_tag: bs4.element.Tag, division: str) -> dict:
    """
    Extracts information about a team, including its stadium and coordinates.

    Args:
        url_team_tag (bs4.element.Tag): The BeautifulSoup tag containing the team information.
        division (str): Team league

    Returns:
        dict: A dictionary with details about the team, including its division, name, stadium,
latitude, and longitude.
    """

    team_info = extract_team_name_url(url_team_tag)
    url_team_wikipedia = next(iter(team_info.values()))
    name_team = next(iter(team_info.keys()))
    search_team = explore_team_page(url_team_wikipedia)
    name_stadium, latitude, longitude = extract_stadium_info(search_team)
    dict_stadium_team = {
        'division': division,
        'equipe': name_team,
        'stade': name_stadium,
        'latitude': latitude,
        'longitude': longitude
    }
    return dict_stadium_team
```

```
def retrieve_all_stadium_from_league(url_list: dict, division: str = "L1") → pd.DataFrame:
    """
    Retrieves information about all stadiums in a league.

    Args:
        url_list (dict): A dictionary mapping divisions to their Wikipedia URLs.
        division (str): The division for which to retrieve stadium information.

    Returns:
         pd.DataFrame: A DataFrame containing information about the stadiums in the specified
division.
    """
    page = retrieve_page(url_list[division])
    teams = page.find_all('span', {'class': 'toponyme'})
    all_info = []

    for team in teams:
        all_info.append(extract_team_info(team, division))

    stadium_df = pd.DataFrame(all_info)
    return stadium_df


# URLs for different divisions
url_list = {
    "L1": "http://fr.wikipedia.org/wiki/Championnat_de_France_de_football_2019-2020",
    "L2": "http://fr.wikipedia.org/wiki/Championnat_de_France_de_football_de_Ligue_2_2019-2020"
}

# Retrieve stadiums information for Ligue 1
stades_ligue1 = retrieve_all_stadium_from_league(url_list, "L1")
stades_ligue2 = retrieve_all_stadium_from_league(url_list, "L2")

stades = pd.concat(
    [stades_ligue1, stades_ligue2]
)
```

```
stades.head(5)
```

	division	equipe	stade	latitude	longitude
0	L1	Paris Saint-Germain Football Club	Parc des Princes	48.8413634	2.2530693
1	L1	LOSC Lille	Stade Pierre-Mauroy	50.611962	3.130631
2	L1	Olympique lyonnais	Parc Olympique lyonnais	45.7652477	4.9818707
3	L1	Association sportive de Saint-Étienne	Stade Geoffroy-Guichard	45.460856	4.390344
4	L1	Olympique de Marseille	Stade Vélodrome	43.269806	5.395922

Tous les éléments sont en place pour faire une belle carte à ce stade. On va utiliser `folium` pour celle-ci, qui est présenté dans la partie visualisation.

5.2 Carte des stades avec `folium`

```
import geopandas as gpd
import folium

stades = stades.dropna(subset = ['latitude', 'longitude'])
stades['latitude'] = stades['latitude'].astype(float)
stades['longitude'] = stades['longitude'].astype(float)
stadium_locations = gpd.GeoDataFrame(
    stades, geometry = gpd.points_from_xy(stades.longitude, stades.latitude)
)

center = stadium_locations[['latitude', 'longitude']].mean().values.tolist()
sw = stadium_locations[['latitude', 'longitude']].min().values.tolist()
ne = stadium_locations[['latitude', 'longitude']].max().values.tolist()

m = folium.Map(location = center, tiles='openstreetmap')

# I can add marker one by one on the map
for i in range(0,len(stadium_locations)):
    folium.Marker(
        [stadium_locations.iloc[i]['latitude'], stadium_locations.iloc[i]['longitude']],
        popup=stadium_locations.iloc[i]['stade']
    ).add_to(m)

m.fit_bounds([sw, ne])
```

La carte obtenue doit ressembler à la suivante :

6 Récupérer des informations sur les pokemons

Le prochain exercice pour mettre en pratique le *web scraping* consiste à récupérer des informations sur les pokemons à partir du site internet pokemondb.net.

6.1 Version non guidée

! Important

Comme pour Wikipedia, ce site demande à `request` d'indiquer un paramètre pour contrôler l'*user-agent*. Par exemple,

```
requests.get(... , headers = {'User-Agent': 'Mozilla/5.0'})
```

Exercice 2 : Les pokémon (version non guidée)

Pour cet exercice, nous vous demandons d'obtenir différentes informations sur les pokémons :

1. les informations personnelles des **893** pokemons sur le site internet pokemondb.net. Les informations que nous aimerions obtenir au final dans un `DataFrame` sont celles contenues dans 4 tableaux :
   - Pokédex data
   - Training
   - Breeding
   - Base stats
2. Nous aimerions que vous récupériez également les images de chacun des pokémons et que vous les enregistriez dans un dossier.

- Petit indice : utilisez les modules `request` et `shutil`
- Pour cette question, il faut que vous cherchiez de vous même certains éléments, tout n'est pas présent dans le TD.

Pour la question 1, l'objectif est d'obtenir le code source d'un tableau comme celui qui suit (Pokémon Nincada).

Pokédex data National № 290 Type Bug Ground Species Trainee Pokémon Height 0.5 m (1′08″) Weight 5.5 kg (12.1 lbs) Abilities 1. Compound EyesRun Away (hidden ability) Local № 042 (Ruby/Sapphire/Emerald)111 (X/Y — Central Kalos)043 (Omega Ruby/Alpha Sapphire)104 (Sword/Shield)

Training EV yield 1 Defense Catch rate 255 (33.3% with PokéBall, full HP) Base Friendship 70 (normal) Base Exp. 53 Growth Rate Erratic

Breeding Egg Groups Bug Gender 50% male, 50% female Egg cycles 15 (3,599–3,855 steps)

Base stats

HP 31

172 266 Attack 45

85 207 Defense 90

166 306 Sp. Atk 30

58 174 Sp. Def 30

58 174 Speed 40

76 196 Total 266 Min Max

Pour la question 2, l'objectif est d'obtenir les images des pokémons.

6.2 Version guidée

Les prochaines parties permettront de faire l'exercice ci-dessus étape par étape, de manière guidée.

Nous souhaitons tout d'abord obtenir les informations personnelles de tous les pokemons sur pokemondb.net.

Les informations que nous aimerions obtenir au final pour les pokemons sont celles contenues dans 4 tableaux :

- Pokédex data
- Training
- Breeding

- Base stats

Nous proposons ensuite de récupérer et afficher les images.

6.2.a Etape 1: constituer un DataFrame de caractéristiques

> Exercice 2b : Les pokémons (version guidée)
>
> Pour récupérer les informations, le code devra être divisé en plusieurs étapes :
>
> 1. Trouvez la page principale du site et la transformer en un objet intelligible pour votre code. Les fonctions suivantes vous seront utiles :
>   - `requests.get`
>   - `bs4.BeautifulSoup`
> 2. A partir de ce code, créer une fonction qui permet de récupérer le copntenu page d'un pokémon à partir de son nom. Vous pouvez nommer cette fonction `get_name`.
> 3. À partir de la page de `bulbasaur`, obtenez les 4 tableaux qui nous intéressent :
>   - on va chercher l'élément suivant : `('table', { 'class' : "vitals-table"})`
>   - puis stocker ses éléments dans un dictionnaire
> 4. Récupérez par ailleurs la liste de noms des pokémons qui nous permettra de faire une boucle par la suite. Combien trouvez-vous de pokémons ?
> 5. Écrivez une fonction qui récupère l'ensemble des informations sur les dix premiers pokémons de la liste et les intègre dans un `DataFrame`.

À l'issue de la question 3, vous devriez obtenir une liste de caractéristiques proche de celle-ci :

La structure est ici en dictionnaire, ce qui est pratique.

Enfin, vous pouvez intégrer les informations des dix premiers pokémons à un `DataFrame`, qui aura l'aspect suivant :

	National №	name	Type	Species	Height	Weight	Abilities	Local №	EV yield	Catch rate	...	Growth Rate	Egg Groups	Gender	Egg cycles	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed
0	0001	bulbasaur	Grass Poison	Seed Pokémon	0.7 m (2′04″)	6.9 kg (15.2 lbs)	1. Overgrow Chlorophyll (hidden ability)	0001 (Red/Blue/Yellow) 0226 (Gold/Silver/Crysta...	1 Sp. Atk	45 (5.9% with PokéBall, full HP)	...	Medium Slow	Grass, Monster	87.5% male, 12.5% female	20(4,884–5,140 steps)	45	49	49	65	65	45
1	0002	ivysaur	Grass Poison	Seed Pokémon	1.0 m (3′03″)	13.0 kg (28.7 lbs)	1. Overgrow Chlorophyll (hidden ability)	0002 (Red/Blue/Yellow) 0227 (Gold/Silver/Crysta...	1 Sp. Atk, 1 Sp. Def	45 (5.9% with PokéBall, full HP)	...	Medium Slow	Grass, Monster	87.5% male, 12.5% female	20(4,884–5,140 steps)	60	62	63	80	80	60
2	0003	venusaur	Grass Poison	Seed Pokémon	2.0 m (6′07″)	100.0 kg (220.5 lbs)	1. Overgrow Chlorophyll (hidden ability)	0003 (Red/Blue/Yellow) 0228 (Gold/Silver/Crysta...	2 Sp. Atk, 1 Sp. Def	45 (5.9% with PokéBall, full HP)	...	Medium Slow	Grass, Monster	87.5% male, 12.5% female	20(4,884–5,140 steps)	80	82	83	100	100	80
3	0004	charmander	Fire	Lizard Pokémon	0.6 m (2′00″)	8.5 kg (18.7 lbs)	1. Blaze Solar Power (hidden ability)	0004 (Red/Blue/Yellow) 0229 (Gold/Silver/Crysta...	1 Speed	45 (5.9% with PokéBall, full HP)	...	Medium Slow	Dragon, Monster	87.5% male, 12.5% female	20(4,884–5,140 steps)	39	52	43	60	50	65
4	0005	charmeleon	Fire	Flame Pokémon	1.1 m (3′07″)	19.0 kg (41.9 lbs)	1. Blaze Solar Power (hidden ability)	0005 (Red/Blue/Yellow) 0230 (Gold/Silver/Crysta...	1 Sp. Atk, 1 Speed	45 (5.9% with PokéBall, full HP)	...	Medium Slow	Dragon, Monster	87.5% male, 12.5% female	20(4,884–5,140 steps)	58	64	58	80	65	80

6.2.b Etape 2: récupérer et afficher des photos de Pokémon

Nous aimerions que vous récupériez également les images des 5 premiers pokémons et que vous les enregistriez dans un dossier.

> Exercice 2b : Les pokémons (version guidée)
>
> - Les URL des images des pokémons prennent la forme « *https://img.pokemondb.net/artwork/{pokemon}.jpg* ». Utilisez les modules `requests` et `shutil` pour télécharger et enregistrer localement les images.
> - Importez ces images stockées au format JPEG dans `Python` grâce à la fonction `imread` du package `skimage.io`.

```
!pip install scikit-image
```



Figure 1. – Les premiers pokemon du Pokédex

7 `Selenium` : mimer le comportement d'un utilisateur internet

Jusqu'à présent, nous avons raisonné comme si nous connaissions toujours l'URL qui nous intéresse. De plus, les pages que nous visitons sont « **statiques** », elles ne dépendent pas d'une action ou d'une recherche de l'internaute.

Nous allons voir à présent comment nous en sortir pour remplir des champs sur un site *web* et récupérer ce qui nous intéresse. La réaction d'un site *web* à l'action d'un utilisateur passe régulièrement par l'usage de `JavaScript` dans le monde du développement *web*. Le *package* Selenium permet de reproduire, depuis un code automatisé, le comportement manuel d'un utilisateur. Il permet ainsi d'obtenir des informations du site qui ne sont pas dans le code `HTML` mais qui apparaissent uniquement à la suite de l'exécution de script `JavaScript` en arrière-plan.

`Selenium` se comporte comme un utilisateur *lambda* sur internet : il clique sur des liens, il remplit des formulaires, etc.

7.1 Premier exemple en scrapant un moteur de recherche

Dans cet exemple, nous allons essayer d'aller sur le site de Bing Actualités et entrer dans la barre de recherche un sujet donné. Pour tester, nous allons faire une recherche avec le mot-clé « **Trump** ».

L'installation de `Selenium` nécessite d'avoir `Chromium` qui est un navigateur Google Chrome minimaliste. La version de chromedriver doit être `⩾ 2.36` et dépend de la version de `Chrome` que vous avez sur votre environnement de travail. Pour installer cette version minimaliste de `Chrome` sur un environnement `Linux`, vous pouvez vous référer à l'encadré dédié.

! Installation deInstalling Selenium

Sur `Colab`, vous pouvez utiliser les commandes suivantes :

```
!sudo apt-get update
!sudo apt install -y unzip xvfb libxi6 libgconf-2-4 -y
!sudo apt install chromium-chromedriver -y
!cp /usr/lib/chromium-browser/chromedriver /usr/bin
```

Si vous êtes sur le `SSP Cloud`, vous pouvez exécuter les commandes suivantes :

```
!wget https://dl.google.com/linux/direct/google-chrome-stable_current_amd64.deb -O /tmp/
chrome.deb
!sudo apt-get update
```

```
!sudo -E apt-get install -y /tmp/chrome.deb
!pip install chromedriver-autoinstaller selenium

import chromedriver_autoinstaller
path_to_web_driver = chromedriver_autoinstaller.install()
```

Vous pouvez ensuite installer `Selenium`. Par exemple, depuis une cellule de `Notebook` :

En premier lieu, il convient d'initialiser le comportement de `Selenium` en répliquant les paramètres du navigateur. Pour cela, on va d'abord initialiser notre navigateur avec quelques options :

```
import time

from selenium import webdriver
from selenium.webdriver.common.keys import Keys

chrome_options = webdriver.ChromeOptions()
chrome_options.add_argument('--headless')
chrome_options.add_argument('--no-sandbox')
chrome_options.add_argument('--disable-dev-shm-usage')
#chrome_options.add_argument('--verbose')
```

Puis on lance le navigateur :

```
from selenium.webdriver.chrome.service import Service
service = Service(executable_path=path_to_web_driver)

browser = webdriver.Chrome(
    service=service,
    options=chrome_options
)
```

On va sur le site de `Bing Actualités`, et on lui indique le mot-clé que nous souhaitons chercher. En l'occurrence, on s'intéresse aux actualités de Donald Trump. Après avoir inspecté la page depuis les outils de développement du navigateur, on voit que la barre de recherche est un élément du code appelé `q` (comme *query*). On va ainsi demander à `selenium` de chercher cet élément:

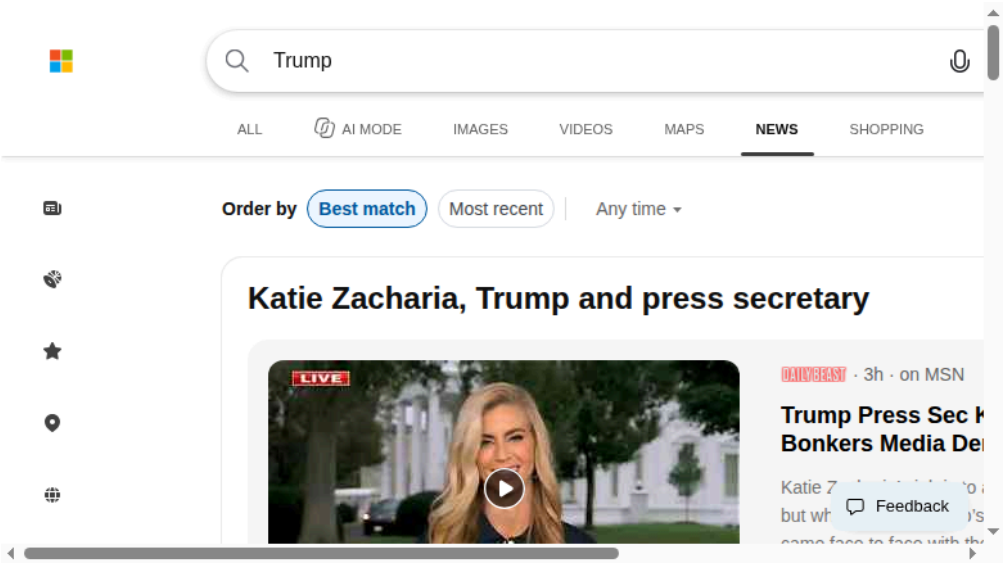
```
browser.get('https://www.bing.com/news')
```

```
search = browser.find_element("name", "q")
print(search)
print([search.text, search.tag_name, search.id])

# on envoie à cet endroit le mot qu'on aurait tapé dans la barre de recherche
search.send_keys("Trump")

search_button = browser.find_element("xpath", "//input[@id='sb_form_go']")
search_button.click()
```

`Selenium` permet de capturer l'image qu'on verrait dans le navigateur avec `get_screenshot_as_png`. Cela peut être utile pour vérifier qu'on a fait la bonne action :



Enfin, on peut extraire les résultats. Plusieurs méthodes sont disponibles. La méthode la plus pratique, lorsqu'elle est disponible, est d'utiliser le `XPath` qui est un chemin non ambigu pour accéder à un élément. En effet, plusieurs éléments peuvent partager la même classe ou le même attribut, ce qui peut faire qu'une recherche de ce type renvoie plusieurs échos. Pour déterminer le `XPath` d'un objet, les outils de développeur de votre site *web* sont pratiques. Par exemple, sous `Firefox`, une fois que vous avez trouvé un élément dans l'inspecteur, vous pouvez faire `click droit > Copier > XPath`.

Enfin, pour mettre fin à notre session, on demande à `Python` de quitter le navigateur:

```
browser.quit()
```

On a obtenu les résultats suivants :

```
['https://www.newsweek.com/trump-allies-ousted-and-under-pressure-weeks-before-midterms-12579502', 'https://www.foxnews.com/outkick-sports/trump-hosted-dinner-stephen-smith-kid-rock-bill-maher', 'https://www.msn.com/en-us/news/other/donald-trump-jr-slams-bayh-in-indiana-secretary-of-state-race-after-grim-poll-for-gop/ar-AA2eppqW?ocid=BingNewsVerp', 'https://www.msn.com/en-us/news/other/trump-says-zacharia-should-reconsider-white-house-spokeswoman-role/ar-AA2eoviT?ocid=BingNewsVerp', 'https://www.cleveland.com/politics/2026/10/trump-takes-rigged-polls-to-a-new-level-with-damaging-gop-theory.html', 'https://www.msn.com/en-us/news/other/trump-says-zero-chance-hes-replacing-chief-of-staff-susie-wiles/ar-AA2eoMOM?ocid=BingNewsVerp', 'https://www.ajc.com/politics/2026/10/trump-in-georgia-friday/']
```

Les autres méthodes utiles de `Selenium`:

Méthode	Résultat
`find_element(****).click()`	Une fois qu'on a trouvé un élément réactif, notamment un bouton, on peut cliquer dessus pour activer une nouvelle page
`find_element(****).send_keys("toto")`	Une fois qu'on a trouvé un élément, notamment un champ où s'authentifier, on peut envoyer une valeur, ici *« toto »*.

7.2 Exercice supplémentaire

Pour découvrir une autre application possible du *web scraping*, vous pouvez également vous lancer dans le sujet 5 de l'édition 2023 d'un hackathon non compétitif organisé par l'Insee :

- Sur `Github`
- Sur le `SSPCloud`

Le contenu de la section NLP du cours pourra vous être utile pour la seconde partie du sujet !

Informations additionnelles

i Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru==0.7.3",
    "markdown≥3.8",
    "nbclient≥0.10.0",
    "nbformat≥5.10.4",
    "nltk≥3.9.1",
    "openpyxl≥3.1.5",
    "pandas≥3.0",
    "pip≥25.1.1",
    "plotly≥6.1.2",
    "plotnine≥0.15",
    "polars≥1.8.2",
    "py7zr≥1.1.3",
    "pyarrow≥17.0.0",
    "pynsee≥0.1.8",
    "python-dotenv≥1.0.1",
```

```
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.